

Formatting Cout Output in C++ using iomanip

Creating cleanly formatted output is a common programming requirement--it improves your user interface and makes it easier to read any debugging messages that you might print to the screen. In C, formatted output works via the `printf` statement, but in C++, you can create nicely formatted output to streams such as `cout`. This tutorial covers a set of basic I/O manipulations possible in C++ from the `iomanip` header file. Note that all of the functions in the `iomanip` header are inside the `std` namespace, so you will need to either prefix your calls with `"std::"` or put `"using namespace std;"` before using the functions.

Dealing with Spacing Issues using iomanip

A principle aspect of nicely formatted output is that the spacing looks right. There aren't columns of text that are too long or too short, and everything is appropriately aligned. This section deals with ways of spacing output correctly.

Setting the field width with setw

The `std::setw` function allows you to set the minimum width of the next output via the insertion operator. `setw` takes, one argument, the width of the next output (insertion), an integer. If the next output is too short, then spaces will be used for padding. There is no effect if the output is longer than the width--note that the output won't be truncated. The only strange thing about `setw` is that its return value must be inserted into the stream. The `setw` function has no effect if it is called without reference to a stream. A simple example is

```
using namespace std;
cout<<setw(10)<<"ten"<<"four"<<"four";
```

The output from the above would look like this:

```
ten      fourfour
```

```
len      four four
```

Note that since `setw` takes an argument, at runtime it would be possible to specify the width of a column of output so that it is slightly wider than the longest element of the column.

You might wonder whether it is possible to change the padding character. It turns out that yes, you can, by using the `setfill` function, which takes a character to use for the padding. Note that `setfill` should also be used as a stream manipulator only, so it must be inserted into the stream:

```
cout<<setfill(' -')<<setw(80)<<"-"<<endl;
```

The above code sets the padding character to a dash, the width of the next output to be at least 80 characters, and then outputs a dash. This results in the rest of the line being filled with dashes too. The output would look like this:

```
- - - - -
```

Note that the pad character is changed until the next time you call `setfill` to change it again.

Aligning text with iomanip

It's possible to specify whether output is left or right aligned by using the manipulator flags that are part of `ios_base`. In particular, it is possible to specify that output should be either left or right aligned by passing in the stream manipulators `std::left` and `std::right`.

Putting Your Knowledge of iomanip Together

Now that we know how to space and align text, we can correctly print formatted data in columns. For instance, if you had a `struct` containing the names of individuals:

```
using namespace std;

struct person
{
    string firstname;
    string lastname;
};
```

If you then had a vector of persons, then you could output them in a nice way with evenly spaced columns for the first and last name as follows:

```
// given the above code, we could write this
vector<person> people;
// fill the vector somehow

int field_one_width = 0, field_two_width = 0;

// get the max widths

for ( vector<person>::iterator iter = people.begin();
      iter != people.end();
      ++iter )
{
    if ( iter->firstname.length() > field_one_width )
    {
        field_one_width = iter->firstname.length();
    }
}
```

```

    }
    if ( iter->lastname.length() > field_two_width )
    {
        field_two_width = iter->lastname.length();
    }
}

// print the elements of the vector
for ( vector<person>::iterator iter = people.begin();
      iter != people.end();
      ++iter )
{
    cout<<setw(field_one_width)<<left<<iter->firstname;
    cout<<" ";
    cout<<setw(field_two_width)<<left<<iter->lastname;
}

```

Note that the space output between the two fields wasn't strictly necessary because we could have added it by changing the first call to setw to set the width to one more than the longest first name (since it would use a space as the padding for the extra character).

Printing Numbers

Another challenge in creating nice output is correctly formatting numbers; for instance, when printing out a hexadecimal value, it would be nice if it were preceded by the "0x" prefix. More generally, it's nice to correctly set the number of trailing zeros after a decimal place.

Setting the precision of numerical output with setprecision

The setprecision function can be used to set the maximum number of digits that are displayed for a number. Like setw, it should be inserted into the stream. In fact, its usage is very similar to setw in all respects. For instance, to print the number 2.71828 to 3 decimal places:

```
std::cout << setprecision(3) << 2.71828;
```

Note that setprecision will change the precision until the next time it is passed into a given stream. So changing the above example to also print out 1.412 would result in the output of

```
2.71 1.41
```

Output in different bases

In computer science, frequently numbers need to be printed in octal or hexadecimal. The setbase function returns a value that can be passed into a stream to set the base of numbers to either base 8, 10, or 16. The input number is still read as a number in base ten, but it is printed in the given base. For instance,

```
std::cout << setbase(16) << 32;
```

will print out "20", which is 32 written in base 16. Note that you can use dec, oct, and hex as shorthand for setbase(10), setbase(8), and setbase(16) respectively when inserting into a stream. If you wish to include an indication of the base along with the printed number, you can use the setiosflags function, again passed into a stream, with an input of ios_base::showbase. Using the ios_base::showbase flag will append a "0x" in front of hexadecimal numbers and a 0 in front of octal numbers. Decimal numbers will be printed as normal.

```
std::cout <<
```

```
std::cout<< setbase(16) << 32;
```

This should get you started with the ability to create nicely formatted output in C++ without having to resort to returning to printf!